

Spatial Election Explorer (SEE)

M. August Rossi, Michael Xiao, Emma Ross, Shu Luan

University of Chicago

May 20, 2026

Abstract. We introduce the Spatial Election Explorer (SEE), an interactive web application for simulating ranked-choice elections on the Euclidean plane. By encouraging users to iteratively build, experiment, and explore, SEE helps them develop visual intuition about the properties and behavior of voting rules, while grounding these concepts in real political opinion survey data.

1 Introduction

Imagine a scenario in which the members of a group have each ranked a set of alternatives from most preferred to least preferred. How should this set of rankings be consolidated to a single ranking or to a set of winning alternatives that fairly represents the entire group’s preferences? The most familiar setting for this problem is an election, where voters rank candidates to choose their representatives, but the problem setup applies much more broadly: consider a set of friends deciding what movie to watch or restaurant to visit; a committee choosing what candidates to shortlist or hire; or a panel of judges who have ranked competitors in ice skating or ballroom dance.

There are many algorithms (called *voting rules*) for aggregating these group preferences: how should the group choose which to use? Deliberation about this decision might involve invoking the details of the algorithms, or making appeals to axiomatic properties that such algorithms may or may not satisfy. Non-experts may find such arguments to be overly complex, too abstract, or simply confusing. Methods to visualize the behavior of voting rules can bridge this communication gap and help voters develop intuition about the differences between algorithms.

The core objective of our project is to employ an established election visualization technique, the two-dimensional spatial model, as the basis of an interactive election simulator. Allowing users to build, experiment, and iterate upon their own simulations builds intuition in ways that a set of static plots cannot. Combining visualization with iterative experimentation allows users to both *see* the properties of the various algorithms and to learn the conditions under which these properties emerge.

Our Methodology. To facilitate visualization, we adopt a two-dimensional spatial model of voters’ preferences. Voters and candidates are jointly embedded in the

Euclidean plane, and voters rank candidates in order of increasing distance. Voters therefore prefer candidates that are closer, or “more similar”, to themselves. This model is common in both theoretical [1] and empirical [2] research in computational social science. Further, it has a natural interpretation: the position of voters and candidates might represent their positions on two political issues or on two ideological axes.

In the Spatial Election Explorer (SEE), voters and candidates can be placed on the plane by hand, by sampling from the available distributions, or by mapping survey data to positions on ideological axes. Their positions determine a unique collection of rankings, which can be used as input to any of various voting rules. The winners are displayed, providing the user a visual description of how the distribution of winners differs according to the choice of voting rule.

Audience. The intended audience for our application is threefold:

1. Computational social choice researchers can experiment with various voting rules, testing and refining the intuition they already have for those methods. Visualization may also suggest patterns of behavior in these voting rules that can be axiomatized and studied formally.
2. Students studying social choice theory can use the tool to develop intuition for the behavior of voting rules that supplements their learning in the classroom.
3. Policymakers and policy advocates considering ranked-choice voting can use SEE to compare voting rules and build intuition about how outcomes vary.

Related Work. Our work is primarily inspired by Elkind et al. [3], whose experiments generated plots capturing the distribution of winning candidates under various multi-winner voting rules and across varying underlying distributions of voters and candidates. SEE extends this approach by making it interactive and incorporating real-world survey data.

Another visualization approach is presented by Szufa et al. [4], who created visual “maps” of preference profiles by computing swap distances between ballots and applying the multidimensional scaling (MDS) algorithm. Extending this method to an interactive application would re-

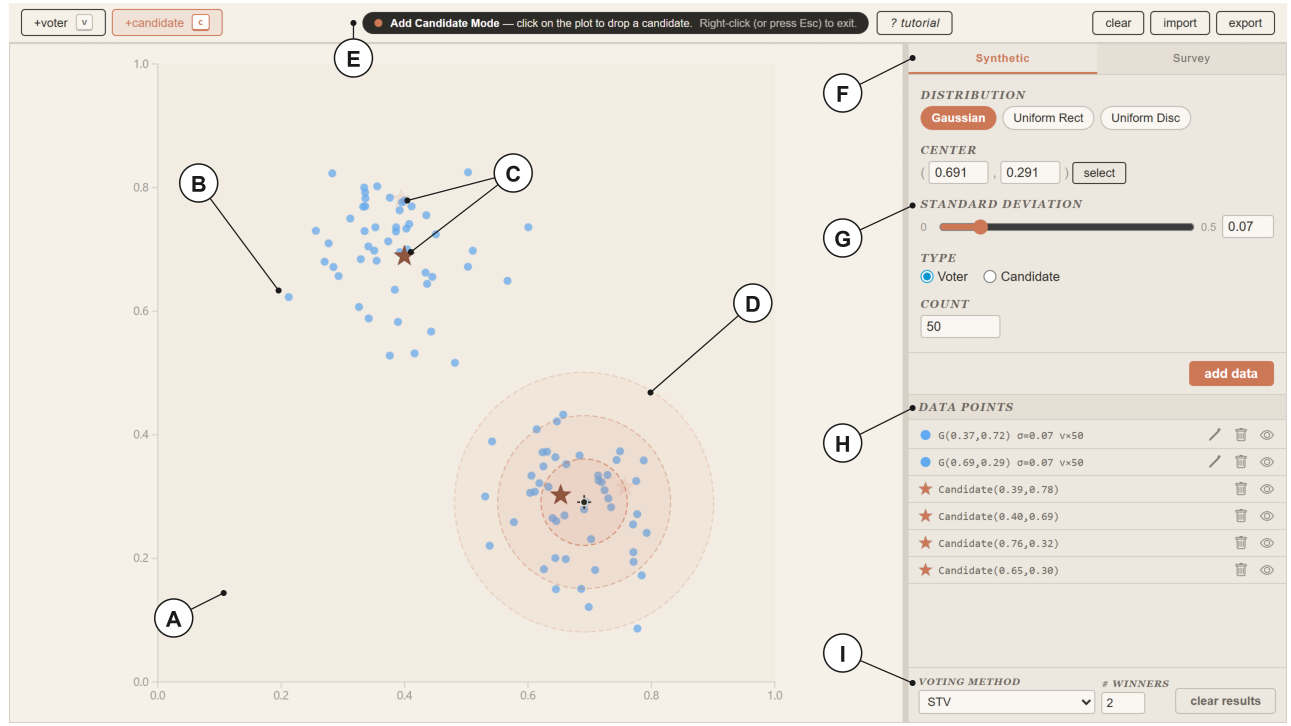


Figure 1: The Spatial Election Explorer (SEE) synthetic data interface. Voters (blue) and candidates (orange) are embedded in a 2D issue space (left); the interface lets the user place points manually, sample from synthetic distributions, load survey-grounded data, manage stacked data layers, and run an election under a chosen voting rule. (A) Abstract 2D election space; (unitless, $[0, 1]$). (B) Voter (blue dot). (C) Candidate (orange star). Winners are rendered at full opacity; eliminated candidates are dimmed in place. (D) Gaussian sampling distribution, drawn as concentric $1\sigma/2\sigma/3\sigma$ contours. (E) Top bar: add voters or candidates by hand, launch the tutorial, import or export the current configuration. (F) Tabs switch between synthetic (§3.2) and survey data exploration (§3.5). (G) Synthetic distribution configuration, and click “add data” to drop the sampled points onto the plot (§3.2). (H) Each manual point or sampled distribution is a stackable, editable layer (§3.3). (I) Choose voting rule (Plurality, Borda, IRV, Bloc Plurality, STV, SNTV) and number of winners, then run the election.

quire some external method for generating election data; the spatial model, on the other hand, consolidates the generation and visualization steps because the position of points *determines* the voters’ rankings.

2 Preliminaries

Spatial Model of Elections. Each voter and candidate has a position in $\mathbb{R}^2$. The pairwise Euclidean distances are computed between all voters and all candidates. A voter’s ballot ranks candidates in order of increasing distance. A collection of such rankings is called a *preference profile*.

Voting Rule. In this context, a *voting rule* is a function that takes as input a preference profile and returns a set of one or more winning alternatives.

Single-Winner Voting Rules.

Plurality. The candidate with the most first-place votes wins.

Borda. Each candidate’s score is the total count

of candidates ranked below them, summed across all ballots. The candidate with the highest score wins.

Instant Runoff Voting (IRV). First-place votes are tallied. The candidate with the fewest votes is eliminated; their supporters’ votes transfer to their next favorite remaining candidate. Repeat until only one candidate remains—they are the winner.

Multi-Winner Voting Rules. Let k be the number of winners, i.e., these rules elect a committee of size k .

Single Non-Transferable Vote (SNTV). First-place votes are tallied. The k highest-scoring candidates are elected.

Bloc Plurality. Each candidate’s score is the number of ballots in which they appear in the top k positions. The k highest-scoring candidates are elected.

k -Borda. The k candidates with the highest Borda scores are elected.

Single Transferable Vote (STV). Let n be the number of voters. First-place votes are tallied to give initial candidate scores. Iterate the following process:

if any candidate’s score s is greater than $q = \lfloor \frac{n}{k+1} \rfloor$, elect them, reweight each supporting voter’s ballot by a factor of $\frac{s-q}{s}$, and transfer their support to their next-favorite remaining candidate. If no candidate is above the threshold, eliminate the candidate with the lowest score and redistribute their support.

Only one candidate can be elected at a time: if multiple candidates are simultaneously above threshold, the candidate with the greater score is elected and their support is redistributed before any other candidates are elected.

For all voting rules, ties are broken arbitrarily.

3 Application Design

SEE provides an interactive plot alongside a sidebar that guides users from first contact—via a tutorial—through placing voters and candidates, running elections under various voting rules, and optionally grounding the simulation in real political survey data.

3.1 Tutorial

The tutorial serves to orient the user, introduce the voting rules, and set user expectations for the allowable interaction patterns in the application.

It begins by defining the spatial model, contextualizing the axes as representing a policy or ideology space, and emphasizing the key assumption: only the distance between voters and candidates matters in determining how voters will fill out their ballots.

Next, it explains the application and the primary ways of using it via a series of examples that introduce Plurality, Borda, IRV, and STV. At each step, the tutorial shows an example plot and asks the user to make a prediction of which candidate will win.

The Plurality example demonstrates the well-known “spoiler effect”, in which a majority bloc of voters is shut out of representation because they split their votes across multiple candidates while the minority bloc is unified in their support of a single candidate.

The IRV example shows how runoff voting can avoid the spoiler effect: the plot is set up the same as in the Plurality example, but, in this case, one of the candidates supported by the majority bloc wins the election.

The Borda example explains how Borda scores are calculated, then demonstrates Borda’s tendency to elect centrist “consensus” candidates.

Finally, the multi-winner example contrasts Bloc Plurality with STV. It shows two blocs of voters, one slightly larger than the other, with two candidates per voter bloc. In Bloc Plurality, the larger bloc can sweep the election, securing both seats on the committee, shutting out the other bloc entirely. However, in STV, each bloc gets one seat on the committee—a more proportional outcome.

3.2 Input

On the plot, voters are blue dots, and candidates are orange stars. There are four primary methods for placing voters and candidates on the plot.

- **Manual Input.** The user can select to add a candidate or a voter, then click anywhere in the plot to add a point at that location.
- **Synthetic Sampling.** In the sidebar, the user can choose from any of the available distributions. They can set the distribution center by dragging it on the plot, or specifying the desired coordinate position in an input box. They can set distribution parameters with sliders or by entering the desired value in an input box. Then, they can set whether to sample voters or candidates, and set the number of points to be sampled. Finally, clicking “add data” adds these points to the plot.

By stacking sets of samples drawn from different distributions, the user can easily create multiple clusters of voters or candidates. As in [3], the supported distributions are the Gaussian, the Uniform Rectangle, and the Uniform Disc.

- **Survey Data.** Upon selecting the “Survey” tab in the sidebar, the user can select two ideology or political issue axes to serve as the axes of the plot. Then, they can view the distribution of political opinion survey respondents across these axes and, by answering the survey questions, can simulate the position of a candidate with specific views. Additionally, users can rank the survey questions used to inform each axis to see how election outcomes differ when voter priorities change. Methodology details are described in Section 3.5.
- **Data Upload.** If a user has exported a JSON file of the candidate and voter points, or survey respondent points, from a previously-generated plot, they may upload that data.

3.3 Plot Management

Every point manually added and every distribution sampled from appears as a separate line item in the “Data Points” section of the sidebar. The line describes the parameters that created it. For every distribution line, the following interactions are available:

- **Edit.** Highlights the distribution and reveals its settings; any of the settings (including type, count, position, and parameters) can be changed, and when the user clicks “update”, the points are resampled. The user can click “cancel” at any point in this interaction if they want to keep the old settings.
- **Delete.** Removes this distribution.

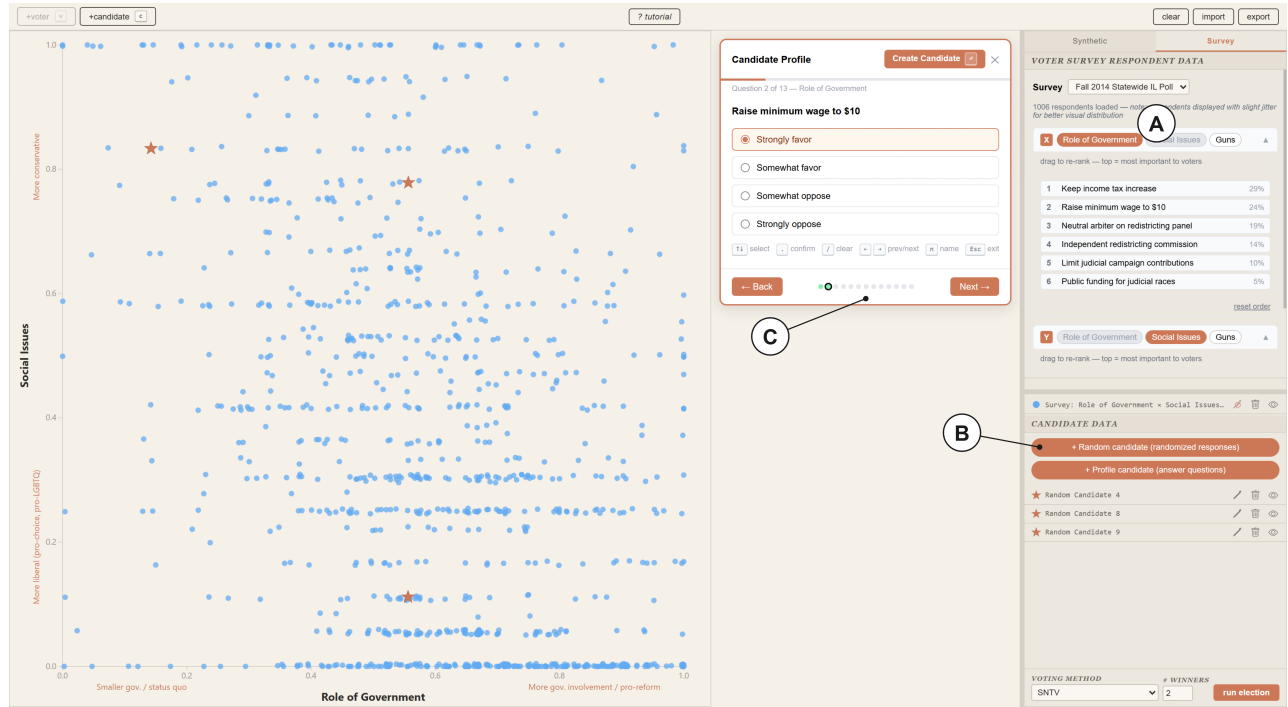


Figure 2: The survey data interface. Voters are loaded from a real political opinion poll (here, the Fall 2014 Statewide IL Poll, 1006 respondents) and projected onto two user-chosen ideology axes — in this example, Role of Government $\times$ Social Issues. Each axis is a weighted aggregate of several survey questions, re-rankable by the user. Candidates can be added either by randomized survey response or by answering the same questions interactively. (A) Axis selection and issue ranking panel. Defines the issue space we are investigating in. (B) +Random candidate creates a candidate point with a random profile. (C) +Profile candidate creates a candidate point with a manually specified profile.

- **Hide.** Hides this distribution; it is temporarily disabled for any election runs, but stays listed in the sidebar to be easily re-enabled later.

For any line item representing a manually-placed candidate or voter, only the Delete and Hide options are available.

3.4 Running Elections

At the bottom of the sidebar, the user can choose their desired voting rule, select the number of winners, and run the election. Running the election updates the plot, labeling the winning candidate(s).

3.5 Survey Data

The Fall 2014 Simon Poll [5], conducted by the non-partisan Paul Simon Public Policy Institute, interviewed 1,006 registered voters across Illinois from September 23–October 15, 2014. The survey asks respondent for demographic details—such as sex, race, ZIP code, income level, education level, and religiosity; whether and for whom they plan to vote in the upcoming election; and for their opinions on political issues including government reform, LGBTQ+ rights, abortion rights, gun control laws, tax-

ation, government budget, public education, police, and public policy.

From this survey, we identified three political topics to use as axes: “Role of Government”, “Social Issues”, and “Guns”. Each topic cluster was validated through pairwise cross-tabulation analysis: for each pair of questions within a topic, we computed the conditional probability that a respondent holding a conservative (or liberal) position on one question also held a consistent position on the other. A question was included only if all pairwise conditional probabilities exceeded 50% in both directions. All question pairs within the final three topics passed this threshold (ranging from 51–79%); four questions were excluded.

The questions in the survey were originally on 4-point Likert scale (strongly favor to strongly oppose), 3-point scale, and binary scale. To normalize, we recoded all scales to a 0-1 range before scoring. Question weights are assigned by linear decay based on the user’s ranking—the top-ranked question receives the most weight, with each subsequent question receiving proportionally less.

3.6 Implementation

The application front end is built with Svelte and D3 in JS, with Vite as the server. The election algorithms are implemented in Python and compiled to Web Assembly via Pyodide so that they can be run in browser.

4 Future Directions

There are a number of natural ways to improve the application or expand its scope.

More Voting Rules. Elkind et al. [3] also examine Chamberlin-Courant (CC) and Monroe, multi-winner voting rules noted for their proportionality properties. These or any of the many methods proposed in the rich social choice literature could be worth inclusion.

Scaling to more voters and candidates. To handle more voters and candidates, two sets of changes are necessary:

1. The Python compute layer should be replaced with higher-performance code written in Rust or C++, which would significantly reduce run times for heavy loads.
2. As the number of voters and candidates grows large, the visual representation must adapt. Rather than plotting all points, the application could, like [3], compute a fine 2D histogram, encoding frequency as color intensity. Methods for choosing representative samples from the dataset, or for estimating and visualizing density, should also be explored.

Additionally, we could replace D3 with a higher-performance visualization library that supports the WebGL API, such as Apache ECharts, deck.gl, or SciChart.js.

Representing Uncertainty in Survey Data. For simplicity, the current implementation of the survey data plots all survey respondents. It might make more sense to sample from the distribution, taking into account (and perhaps visually representing) the uncertainty inherent in the survey data.

More Survey Data. As proof of concept, we have included a single source of survey data. The application would benefit from more options and more data sources.

References

- [1] J. M. Enelow and M. J. Hinich, *The Spatial Theory of Voting* (Cambridge Books 9780521275156). Cambridge University Press, Enero-Abr 1984.
- [2] N. Boehmer, P. Faliszewski, Ł. Janeczko, A. Kaczmarczyk, G. Lisowski, G. Pierczyński, S. Rey, D. Stolicki, S. Szufa, and T. Wąs, *Guide to Numerical Experiments on Elections in Computational Social Choice*, 2024. arXiv: 2402.11765 [cs.GT].
- [3] E. Elkind, P. Faliszewski, J.-F. Laslier, P. Skowron, A. Slinko, and N. Talmon, “What Do Multiwinner Voting Rules Do? An Experiment Over the Two-Dimensional Euclidean Domain,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31, no. 1, Feb. 2017.
- [4] P. Faliszewski, A. Kaczmarczyk, K. Sornat, S. Szufa, and T. Wąs, “Diversity, Agreement, and Polarization in Elections,” in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*, E. Elkind, Ed., Main Track, International Joint Conferences on Artificial Intelligence Organization, Aug. 2023, pp. 2684–2692.
- [5] Paul Simon Public Policy Institute, “Simon Poll, Fall 2014 (state wide),” Paul Simon Public Policy Institute, Southern Illinois University Carbondale, Tech. Rep. 8, 2014.